Operating Systems Lab

Viva Theory Guide

This guide covers the theory most commonly asked in the OS lab viva for programs Q1 to Q8 (Q9 is excluded as requested). Start with the Foundations section — those questions come up no matter which program you are asked about. Then read the section for each program. The last section is a quick ‘X vs Y’ comparison sheet for fast revision.

Foundations (asked in almost every OS viva)

Q. What is an operating system (OS)?

System software that manages the computer's hardware and software resources and acts as an interface between the user/applications and the hardware. It manages processes, memory, files, and I/O devices.

Q. What is a system call?

A request made by a program to the OS kernel to perform a privileged operation it cannot do itself, such as file I/O, creating a process, or allocating memory. It is the doorway between a user program and the kernel.

Q. Difference between a system call and an ordinary (library) function call?

An ordinary/library function runs entirely in user space. A system call switches the CPU from user mode to kernel mode (a 'mode switch') to do privileged work, so it is slower. Example: printf() is a library function that internally uses the write() system call.

Q. What is user mode vs kernel mode?

Kernel mode has full, unrestricted access to hardware and all instructions. User mode is restricted and cannot directly touch hardware. Normal programs run in user mode and must use system calls to ask the kernel (running in kernel mode) for privileged services. This separation protects the system.

Q. What is a process?

A program in execution. It has its own address space (code, data, heap, stack), a unique process ID (PID), CPU register values, and allocated resources.

Q. Difference between a program and a process?

A program is a passive set of instructions stored on disk. A process is an active, running instance of a program, with state and resources. One program can have many processes.

Q. What is a file descriptor?

A small non-negative integer that the OS gives you to refer to an open file (or device/socket). By convention 0 = standard input, 1 = standard output, 2 = standard error. open() returns one.

Q. What are the main UNIX file-I/O system calls?

open, read, write, close, and lseek. These are the low-level 'UNIX file APIs'.

Q. Low-level (system call) I/O vs high-level (stdio) I/O?

Low-level (open/read/write) uses file descriptors, is unbuffered, and goes straight to the kernel. High-level C standard I/O (fopen/fread/fprintf) uses FILE* streams and is buffered in user space for efficiency. The lab programs use the low-level calls.

Q1 - Implement ls -l using UNIX file APIs

Q. What does this program do?

It re-creates the 'ls -l' command: it lists every file in the current directory along with its details — file type, permissions, link count, owner, group, size, last-modified time, and name.

Q. What is a directory in UNIX?

A directory is itself a special file that stores a list of entries, where each entry maps a file name to an inode number.

Q. What is an inode?

A data structure that stores all the metadata about a file: its type, permissions, owner, group, size, timestamps, link count, and pointers to its data blocks. Note: the file's NAME is not in the inode (the name lives in the directory).

Q. What do opendir(), readdir(), closedir() do?

opendir() opens a directory for reading and returns a directory stream; readdir() returns the next entry (a struct dirent containing the name d_name); closedir() closes the stream.

Q. What does stat() do and what is in struct stat?

stat() fetches a file's metadata into a struct stat. Key fields: st_mode (file type + permission bits), st_nlink (link count), st_uid (owner id), st_gid (group id), st_size (bytes), and st_mtime (last-modified time).

Q. Difference between stat, lstat and fstat?

stat() follows a symbolic link and reports the target file. lstat() reports the link itself. fstat() does the same as stat() but takes an open file descriptor instead of a path name.

Q. What are the file types in UNIX (the first character of ls -l)?

- (regular file), d (directory), c (character device), b (block device), l (symbolic link), p (FIFO / named pipe), s (socket).

Q. What do macros like S_ISDIR and S_ISREG do?

They test the st_mode field to tell you the file type, e.g. S_ISDIR(st_mode) is true for a directory, S_ISREG for a regular file.

Q. What do the permission bits mean?

There are 9 permission bits in three groups — owner, group, others — each with read (r), write (w), execute (x). For example rwxr-xr-- means owner can read/write/execute, group can read/execute, others can only read.

Q. What does the mode 0644 mean?

It is octal. 6 = rw- (owner), 4 = r-- (group), 4 = r-- (others). So owner can read+write; everyone else can only read.

Q. What does getpwuid() and getgrgid() do?

They convert a numeric user ID into a user name, and a numeric group ID into a group name (looked up in /etc/passwd and /etc/group). ls -l shows names, not numbers, because of these.

Q. What is st_nlink?

The number of hard links pointing to that file (how many directory entries refer to the same inode).

Q2 - Implement cp, rm & mv using UNIX file APIs

Q. What does this program do?

It implements three commands using UNIX system calls: cp (copy a file), mv (move/rename a file), and rm (delete a file).

Q. How does the cp program work?

It opens the source file read-only, opens/creates the destination file write-only, then loops: read() a chunk into a buffer and write() it to the destination until read() returns 0 (end of file). Finally it closes both files.

Q. What does open() return and what are its common flags?

It returns a file descriptor (a small integer), or -1 on error. Common flags: O_RDONLY, O_WRONLY, O_RDWR, O_CREAT (create if it doesn't exist), O_TRUNC (empty it), O_APPEND (write at the end).

Q. What do read() and write() return?

read() returns the number of bytes actually read (0 means end of file, -1 means error). write() returns the number of bytes written, or -1 on error.

Q. How does the mv program work?

It calls link(old, new) to create a second name (a hard link) for the same file, then unlink(old) to remove the original name. The result is that the file has been renamed/moved.

Q. What is a hard link?

An additional directory entry (name) that points to the same inode/data as an existing file. The file's data survives as long as at least one hard link (link count > 0) remains.

Q. What does unlink() do?

It removes a directory entry and decreases the file's link count. When the link count reaches 0 (and no process still has the file open), the file's data is actually freed. The rm program is just unlink().

Q. Difference between a hard link and a soft (symbolic) link?

A hard link is another name for the same inode; it cannot cross filesystems and cannot link directories. A soft link is a separate small file that stores the path to the target; it can cross filesystems but breaks if the target is deleted.

Q. Why does this mv only work within one filesystem?

Because it uses link() (a hard link), and hard links cannot span different filesystems (inode numbers are local to each filesystem). The real mv command falls back to copy-then-delete when crossing filesystems.

Q. What is perror() and errno?

errno is a global variable the OS sets to a code describing the last error. perror() prints a readable message for that error (e.g. 'open: No such file or directory').

Q3 - fork, execve, wait, getpid, exit

Q. What does this program do?

It demonstrates process control: the parent uses fork() to create a child; the child uses exec to replace itself with the binary-search program; the parent uses wait() to wait for the child and then checks how it exited.

Q. What is fork()?

A system call that creates a new process (the child) which is an almost-identical copy of the calling process (the parent).

Q. What does fork() return?

It returns twice: in the parent it returns the child's PID (a positive number); in the child it returns 0; and it returns -1 if the fork failed. This is how code tells whether it is the parent or the child.

Q. What is copy-on-write?

An optimization for fork: parent and child initially share the same physical memory pages, and a page is only copied when one of them tries to write to it. This makes fork fast and memory-efficient.

Q. What is the exec family of functions?

Functions (execl, execv, execlp, execvp, execve, execle) that replace the current process's program with a new one. They differ in how you pass arguments (list vs vector), whether they search the PATH, and whether you pass an environment.

Q. Difference between fork and exec?

fork() creates a brand-new process (a duplicate). exec() does NOT create a new process — it loads a different program into the current process, keeping the same PID. They are often used together: fork, then exec in the child.

Q. What does exec return on success?

Nothing — on success exec never returns, because the old program has been completely replaced. It returns -1 only if it fails.

Q. What does wait() do?

It makes the parent block (pause) until a child finishes, then collects the child's exit status and removes the finished child from the process table. This is called 'reaping' the child.

Q. What is a zombie process?

A child that has finished running but whose exit status has not yet been collected by its parent. It still occupies a slot in the process table. Calling wait() removes it.

Q. What is an orphan process?

A child whose parent has already terminated. It is adopted by the init process (PID 1), which becomes its new parent.

Q. What do getpid() and getppid() return?

getpid() returns the current process's own PID; getppid() returns its parent's PID.

Q. What do WIFEXITED and WEXITSTATUS do?

They interpret the status value returned by wait(). WIFEXITED is true if the child terminated normally; WEXITSTATUS extracts the actual exit code the child returned.

Q. What is the init process?

The very first process the kernel starts, with PID 1. It is the ancestor of all other processes and adopts orphaned processes.

Q4 - pthread_create, pthread_join, pthread_exit

Q. What does this program do?

It demonstrates the POSIX threads (pthread) library by creating multiple threads that work on a shared matrix at the same time.

Q. What is a thread?

A lightweight unit of execution inside a process — the smallest sequence of instructions the scheduler can run. A single process can contain many threads.

Q. Difference between a process and a thread?

Threads of the same process share one address space (same code, data, heap, open files), while processes each have separate address spaces. Threads are lighter, faster to create, and faster to switch between.

Q. What do threads share and what is private to each thread?

Shared: code, global/static data, heap, and open file descriptors. Private to each thread: its own stack, CPU registers, program counter, and thread ID.

Q. What does pthread_create() do?

It creates a new thread and starts it running a given function. Its arguments are: a pointer to store the thread ID, thread attributes (usually NULL), the start function, and an argument to pass to that function.

Q. What does pthread_join() do?

It makes the calling thread wait until the specified thread has finished — the thread equivalent of wait(). It can also collect the finished thread's return value.

Q. What does pthread_exit() do?

It terminates the calling thread (and can return a value). Calling it in main() lets the other threads keep running even after main()'s own code has ended.

Q. Why must you compile thread programs with -lpthread?

Because the pthread functions live in the POSIX threads library, which must be linked in. Without -lpthread the program will not link/compile.

Q. What are the advantages of threads over processes?

Faster to create and switch, lower overhead, and easy data sharing because they share memory. Good for concurrency within one application.

Q. What are the disadvantages of threads?

No memory isolation — one buggy thread can crash the whole process; and because they share memory, you must use synchronization (mutexes/semaphores) to avoid race conditions.

Q. Concurrency vs parallelism?

Concurrency means multiple tasks are making progress over the same period (possibly interleaved on one CPU). Parallelism means tasks literally run at the same instant on multiple CPU cores.

Q5 - Dining Philosophers (semaphores + mutex)

Q. What is the Dining Philosophers problem?

A classic synchronization problem: five philosophers sit around a table; between each pair is one fork. A philosopher needs BOTH neighboring forks to eat. It is used to illustrate resource sharing, deadlock, and starvation among concurrent processes.

Q. What is a critical section?

A part of the program that accesses shared resources and must be executed by only one thread/process at a time to avoid incorrect results.

Q. What is a race condition?

A bug where the result depends on the unpredictable timing or interleaving of concurrent accesses to shared data, producing wrong or inconsistent output.

Q. What is mutual exclusion?

The rule that only one thread/process can be inside the critical section at any given time.

Q. What is a semaphore?

An integer variable used for synchronization that can only be changed through two atomic operations: wait (also called P or down) and signal (also called V or up). It can block a process until a resource is free.

Q. What is the difference between a binary and a counting semaphore?

A binary semaphore takes only 0 or 1 (used like a lock). A counting semaphore can take any non-negative value (used to count multiple available instances of a resource).

Q. What do the wait (P) and signal (V) operations do?

wait/P: if the value is greater than 0, decrement it and continue; otherwise block. signal/V: increment the value and wake up a waiting process if any.

Q. What is a mutex?

A mutual-exclusion lock (similar to a binary semaphore) used to protect a critical section. A key idea is ownership: the thread that locks a mutex is the one that should unlock it.

Q. Mutex vs binary semaphore?

A mutex is specifically for mutual exclusion and has ownership (only the locker unlocks). A semaphore is a general signaling tool and can be posted by any thread. They look similar but are used differently.

Q. What is deadlock?

A situation where a group of processes are each waiting for a resource held by another, so none can ever proceed. Example: every philosopher grabs the left fork and waits forever for the right one.

Q. What are the four (Coffman) conditions for deadlock?

All four must hold at once:
(1) Mutual exclusion (2) Hold and wait (3) No preemption (4) Circular wait. Breaking any one of them prevents deadlock.

Q. How does this solution avoid deadlock?

A philosopher is only allowed to start eating if BOTH neighbors are not eating (checked in the test() function). Philosophers do not pick up one fork and then wait holding it, which breaks the 'hold and wait' / circular-wait conditions.

Q. What is starvation?

When a process waits indefinitely because other processes keep getting the resource first. It is not deadlocked (others are progressing), it is just never served.

Q. What do sem_init, sem_wait and sem_post do?

sem_init initializes a semaphore to a starting value; sem_wait is the wait/P/down operation; sem_post is the signal/V/up operation.

Q6 - Producer-Consumer (semaphores + mutex)

Q. What is the Producer-Consumer (bounded buffer) problem?

Producers create items and put them into a shared fixed-size buffer; consumers take items out. They must be synchronized so a producer never adds to a full buffer and a consumer never removes from an empty one.

Q. Why does it need two semaphores plus a mutex?

The 'empty' semaphore counts free slots and the 'full' semaphore counts filled slots — these handle the full/empty conditions and block threads when needed. The mutex separately ensures only one thread touches the buffer at a time.

Q. What are the initial values of the semaphores?

empty = buffer size (all slots free at the start), full = 0 (no items yet), and mutex = 1 (unlocked).

Q. What does a producer do, step by step?

wait(empty) to claim a free slot, lock the mutex, write the item into the buffer, unlock the mutex, then signal(full) to announce a new item.

Q. What does a consumer do, step by step?

wait(full) to ensure an item exists, lock the mutex, remove the item, unlock the mutex, then signal(empty) to announce a freed slot.

Q. What goes wrong without synchronization?

Race conditions: the buffer can overflow (producing into a full buffer), underflow (consuming from an empty buffer), or the shared index can get corrupted — leading to lost or wrong data.

Q. Why keep the mutex separate from full/empty?

full and empty handle counting and blocking based on how full the buffer is; the mutex specifically guarantees mutual exclusion while modifying the buffer. They solve two different problems.

Q. What is busy waiting, and do semaphores avoid it?

Busy waiting is repeatedly checking a condition in a loop, which wastes CPU. Semaphores avoid it by putting the waiting process to sleep (blocking) until it is signalled.

Q7 - Reader-Writer (semaphores + mutex)

Q. What is the Readers-Writers problem?

Several processes share data. Readers only read and may do so simultaneously; writers modify the data and need exclusive access. The goal is to allow many concurrent readers while keeping writes exclusive.

Q. How is it different from Producer-Consumer?

Producer-Consumer is about adding to / removing from a shared buffer (counting slots). Readers-Writers is about an access pattern: allow many simultaneous readers but only one writer at a time.

Q. Why can many readers read at once but only one writer write?

Reading does not change the data, so it is safe to share among many readers. Writing changes the data, so it must be exclusive; otherwise readers could see half-updated, inconsistent data.

Q. What is rcount and why is it protected by a mutex?

rcount is the count of readers currently reading. It is shared, so it is protected by a mutex to stop two readers from updating it at the same time and corrupting the count.

Q. What does the 'wr' semaphore do?

It grants exclusive write access. Writers acquire it directly; readers acquire it collectively (through the first/last reader logic) so that writers are blocked while any reader is active.

Q. Why does the first reader lock 'wr' and the last reader unlock it?

So that as long as at least one reader is reading, all writers are kept out. The first reader to arrive locks out writers, and the last reader to leave lets writers back in.

Q. What is reader-preference and what problem does it cause?

This solution is reader-preference: if readers keep arriving continuously, writers can be made to wait forever. That is writer starvation.

Q8 - Synchronization using File Locks

Q. What does this program do?

It synchronizes access to a file between processes using file locks set through fcntl(). One process can lock a region of the file so another process cannot modify it at the same time.

Q. What is file locking and why is it needed?

It is a mechanism to coordinate access to a file (or part of a file) so that multiple processes do not corrupt it by writing simultaneously. It is process-level synchronization.

Q. What is fcntl()?

A system call that performs various control operations on a file descriptor, including setting and querying file locks (using commands F_SETLK, F_SETLKW, F_GETLK).

Q. What is struct flock and its fields?

It describes a lock. l_type = lock type (read/write/unlock); l_whence = the reference point for the offset (SEEK_SET/CUR/END); l_start = starting offset; l_len = length of the region; l_pid = PID of the process holding the lock (filled in by F_GETLK).

Q. What are the lock types?

F_RDLCK = read (shared) lock; F_WRLCK = write (exclusive) lock; F_UNLCK = remove the lock.

Q. Difference between a read lock and a write lock?

Many processes can hold a read (shared) lock on the same region at once. A write (exclusive) lock can be held by only one process and excludes everyone else — the same idea as the Readers-Writers problem.

Q. Difference between F_SETLK, F_SETLKW and F_GETLK?

F_SETLK tries to set the lock and fails immediately (returns -1) if it cannot. F_SETLKW sets the lock but waits (blocks) until it becomes available. F_GETLK only tests whether a lock could be placed and tells you who currently holds it.

Q. What is advisory vs mandatory locking?

Advisory locking (the default) only works if processes cooperate by checking locks before accessing the file. Mandatory locking is enforced by the kernel automatically on every read/write, but is rarely used.

Q. What is lseek()?

A system call that moves the file's read/write position (offset). The 'whence' argument can be SEEK_SET (from the start), SEEK_CUR (from current position), or SEEK_END (from the end).

Q. How does this program demonstrate synchronization?

If one process holds a write lock and a second process tries to lock the same region, the second one is refused (F_SETLK returns -1) and is told the first process's PID — so access is coordinated between the two processes.

Quick 'X vs Y' comparison sheet (fast revision)

Q. Process vs Thread

Process = independent program with its own address space; heavy; isolated. Thread = unit of execution inside a process; shares the process's memory; lightweight; faster to create/switch but not isolated.

Q. System call vs Library function

System call enters the kernel (mode switch) to do privileged work; slower. Library function runs in user space; faster. printf (library) uses write (system call) underneath.

Q. fork vs exec

fork creates a new process (a copy), keeps running the same program. exec replaces the current program with a new one, same PID, no new process.

Q. Semaphore vs Mutex

Mutex = lock for mutual exclusion, has ownership (locker unlocks), value 0/1. Semaphore = signaling mechanism, can be counting (any value), can be posted by any thread.

Q. Binary vs Counting semaphore

Binary = only 0 or 1 (acts like a lock). Counting = any non-negative value (counts multiple available resource instances).

Q. Hard link vs Soft (symbolic) link

Hard link = another name for the same inode; same filesystem only; survives until link count hits 0. Soft link = a file holding a path to the target; can cross filesystems; breaks if target is deleted.

Q. Deadlock vs Starvation

Deadlock = processes permanently stuck, each waiting on another (no one progresses). Starvation = one process waits indefinitely while others keep progressing.

Q. Concurrency vs Parallelism

Concurrency = multiple tasks in progress over the same period, possibly interleaved on one CPU. Parallelism = tasks actually executing at the same instant on multiple cores.

Q. stat vs lstat vs fstat

stat = info about a file by path, follows symlinks. lstat = info about the symlink itself. fstat = info about an already-open file descriptor.

Q. F_SETLK vs F_SETLKW

F_SETLK = try to lock, fail immediately if busy. F_SETLKW = try to lock, wait (block) until it is free.